

Ejercicios Colecciones Kotlin

[Descargar estos ejercicios](#)

Ejercicio 1

Teniendo en cuenta las colecciones vistas en el tema, indica qué tipo de colección sería el más adecuado para cada caso y crea una variable inicializada con un ejemplo. Decide si debe ser **var** o **val** según la mutabilidad que necesites.

1. Colección para almacenar los nombres de los alumnos de una clase, el nombre y cantidad de alumnos no cambiará una vez introducidos en el sistema.
2. Colección para guardar la nota final de cada alumno de un grupo (solo se guarda la nota), de forma que cada alumno tenga asociado un valor numérico de la colección, la nota puede variar a lo largo del curso. (El primer alumno de la lista, tendrá su nota guardada en la primera posición de la colección)
3. Conjunto con los códigos de los ordenadores disponibles en un aula, sin que se repitan. Investiga la colección `setOf`
4. Colección con los ejercicios de programación que debe entregar un estudiante durante el curso, pudiendo añadir y quitar tareas.
5. Relación entre cada día de la semana y la asignatura que se imparten cada día (serán un mínimo de tres).
6. Colección para guardar los idiomas que domina una persona, sin repetir ninguno.
7. Colección con los precios de los productos de un supermercado que cambian cada semana, tanto en cantidad como en valor.
8. Colección para guardar el nombre del usuario y su contraseña en una aplicación, donde el usuario será la clave y la contraseña podrá modificarse.
9. Colección para almacenar el menú de cada día de la semana en el comedor del instituto, usando un tipo **Pair** en un **Map**.
10. Conjunto de palabras clave que describen un proyecto de programación, sin duplicados.

Ejercicio 2

Vamos a crear una de las parte de una aplicación, la que se encarga de la gestión de las contraseñas. Para ello crearemos un tipo **object GestionClaves** (para conseguir la

funcionalidad de una clase estática), el tipo tendrá un campo privado de tipo **MutableMap** **contraseña** con sus dos partes a String (una para el login y la otra para el password), un tipo exception **ClavesException** para los casos inválidos y las funciones necesarias que te permitan añadir, modificar y eliminar el login y password de los usuarios.

Antes de salir de programa se mostrará el contenido del MutableMap, para comprobar que los datos se han gestionado correctamente.

El programa principal podría mostrar un menú como el siguiente:

1. Añade login
2. Modifica password
3. Elimina login
4. Salir



Aviso

Al añadir y eliminar se deberá controlar que el login no sea vacío, al modificar se deberá controlar que el login existe y se pedirá el password antiguo para comprobar que coincide y permitir el cambio.

Ejercicio 3

Vamos a seguir desarrollando el ejercicio de la gestión de series de la entrega anterior. Para ello deberemos usar la colección más adecuada posible para almacenar las series que un supuesto espectador esta visualizando, ya ha visualizado o visualizará en un futuro.

A partir del código anterior, procederás a dar la funcionalidad necesaria para sustituir las salidas por pantalla que teníamos hasta ahora, por la gestión correcta de las series en la colección elegida.

Debes usar las funciones de búsqueda, selección, etc. del paquete **kotlin.collections** para conseguir un código más corto y claro.

Puedes usar los siguientes datos para probar el programa [ListaSeries](#)